

Lesson6

面向对象的三大特称之三：多态 (polymorphism)



主要内容

- 认识多态
 - 多态的必要性
 - 静多态和动多态
 - 多态的实现
 - 多态的优点及运行机制
- 抽象类
 - 认识抽象类
 - 理解抽象方法
 - 理解抽象类引用
- 面向对象设计模式：工厂模式
- 案例分析

什么是多态

- 生活中的多态

- 你能列举出一个多态的生活示例吗？

同一种事物，由于条件不同，产生的结果也不同

- 同一种事物的多种形态

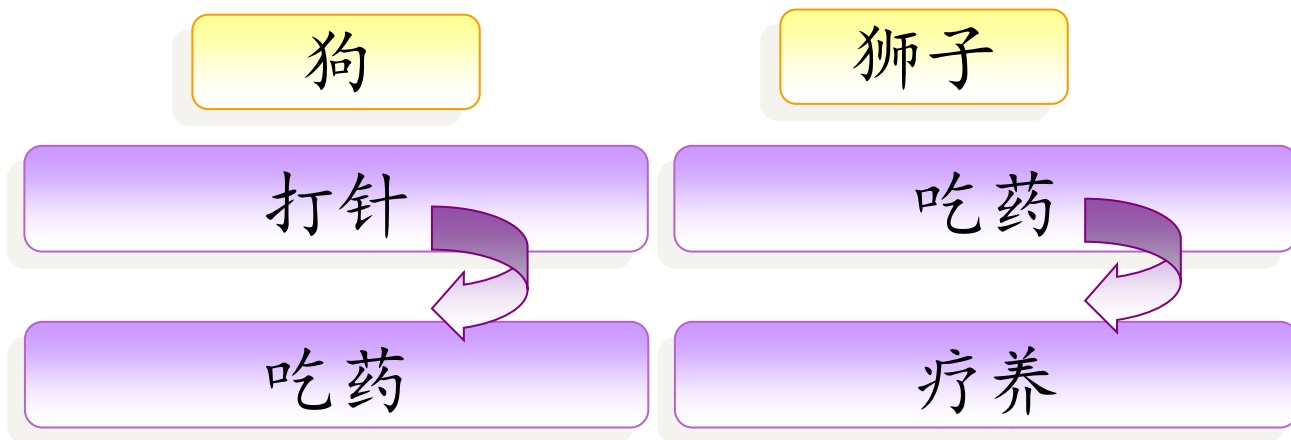
- 程序中的多态

- 同一个函数调用，产生不同的效果

- 同一个变量，代表不同的对象

为什么使用多态？

- 动物生病了，需要医生给动物看病
 - 不同动物看病过程不一样



- 不同动物恢复后状态不一样

传统的代码实现

测试方法

- 编码实现

医生类

```
public class Doctor {  
    public void cure(Dog dog) {  
        if (dog.getHealth() < 50) {  
            dog.setHealth(60);  
            System.out.println("打针、吃药");  
        }  
    }  
    public void cure(Lion lion){  
        if (lion.getHealth() < 50)  
            lion.setHealth(70);  
        System.out.println("吃药、疗养");  
    }  
}
```

... ..

```
Doctor doctor = new  
Doctor();
```

```
doctor.cure(dog);  
doctor.cure(lion);
```

... ..

为什么使用多态？

- 如果又需要给XXX看病，怎么办？
 - 添加XXX类，继承Animal类
 - 修改Doctor类，添加给XXX看病的方法

频繁修改代码，代码可扩展性、可维护性差

使用 **多态** 优化设计

为什么使用多态?

• 使用多态优化后的代码

Dog类

```
public class Dog extends Animal {  
    public void toHospital() {  
        this.setHealth(60);  
        System.out.println("打针、吃药");  
    }  
}
```

1

Lion类

```
public class Lion extends Animal {  
    public void toHospital() {  
        this.setHealth(70);  
        System.out.println("吃药、疗养");  
    }  
}
```

2

医生类

```
public class Doctor {  
    public void cure(Animal animal){  
        if (animal.getHealth() < 50)  
            animal.toHospital();  
    }  
}
```

3

父类引用子类对象

测试方法

```
... ..  
Animal dog = new Dog();  
Doctor doctor = new Doctor();  
doctor.cure(dog);  
Animal lion = new Lion();  
doctor.cure(lion);  
... ..
```

4

为什么使用多态？

- 又要给XXX看病时，只需：
 1. 编写XXX类继承Pet类（旧方案也需要）
 2. 创建XXX类对象（旧方案也需要）
 3. 其他代码不变（不用修改Doctor类）

代码可扩展性、可维护性变好了！

动多态：运行时根据所引用的具体实例来确定调用父类方法还是子类方法

程序中的多态

同一个**引用类型**，使用不同的实例而执行不同操作

```
public class Doctor {  
    public void cure(Animal animal) {  
        if (animal.getHealth() < 50)  
            animal.toHospital();  
    }  
}
```

```
... ..  
Animal dog = new Dog();  
Doctor doctor = new Doctor();  
doctor.cure(dog);  
Lion lion = new Lion();  
doctor.cure(lion);  
... ..
```

多态是面向对象程序设计的另一个**重要特征**，其基本含义是“**拥有多种形态**”，具体指在程序中**用相同的名称来表示不同的含义**。例如：用同一方法名来表示不同的操作。

为什么会出现多态？

- Java 中的引用变量有两个类型：
 - 一个是编译时的类型，一个是运行时的类型，
编译时的类型由声明该变量时使用的类型决定，
运行时的类型由实际赋给该变量的对象决定。
- 如果编译时的类型与运行时的类型不一致
就会出现所谓的多态？

主要内容

- 认识多态
 - 多态的必要性
 - 静多态和动多态
 - 多态的实现
 - 多态的优点及运行机制
- 抽象类
 - 认识抽象类
 - 理解抽象方法
 - 理解抽象类引用
- 面向对象设计模式：工厂模式
- 案例分析

多态性类型

- 多态类型
 - 静态多态性：包括方法的重载；
 - 动态多态性：在编译时不能确定所要调用的方法，只有在运行时才能确定所要调用的方法，又称为运行时的多态性。

静态多态（静多态）

- 静态多态：即在编译时决定调用哪个方法，也称为编译时多态，也称为静态联编，也称为静绑定；
- 静态多态一般是指方法重载（overload）；
- 只要构成了方法重载，就可以认为形成了静态多态的条件；因此，静态多态与是否发生继承没有必然联系。

方法重载 (Overload)

- 方法重载：Java允许在一个类中定义多个同名的方法，但这些方法的参数列表必须不同
— 方法名相同，参数个数、参数类型及参数顺序至少有一个不同
- 重载的目的：一般是为了创建一组要完成相似任务的成员方法。
- 构造方法和静态成员方法都是可以重载，静态成员方法重载后的方法也可以是非静态成员方法。

方法重载 (Overload)

- 对于方法重载，返回值类型与访问权限修饰符可以相同也可以不同，上述两项不能当做判断是否重载的条件。
- 如果一个类中有两个同名方法，其参数列表完全一样，仅仅返回值类型不同，则编译时会产生错误

com.buaa.classEx.MethodOverload

```
public class MethodOverload {  
    public static void main(String args[]) {  
        int a = 51, b = -98, c = 8, d = 191;  
        double u = 25.1, v = -29.8, x = 3.1, y = 89.98;  
        System.out.println("51、-98、8、191四数的最大值是: " + max(a, b, c, d));  
        System.out.println("25.1、-29.8、3.1、89.98四数的最大值是: " + max(u, v, x, y));  
    }  
}
```

```
static int max(int a, int b, int c, int d) {
```

```
    int x, y;  
    x = a > b ? a : b;  
    y = c > d ? c : d;  
    return x > y ? x : y;  
}
```

```
static double max(int a, int b, int c, int d) {  
    double x, y;  
    x = a > b ? a : b;  
    y = c > d ? c : d;  
    return x > y ? x : y;  
}
```

```
/*
```

* 如果一个类中有两个同名方法，其参数列表完全一样，仅仅返回值类型不同，则编译时会产生错误

```
*/
```

```
/*
```

```
static double max(int a, int b, int c, int d) {
```

```
    int x, y;  
    x = a > b ? a : b;  
    y = c > d ? c : d;  
    return x > y ? x : y;  
}
```

```
*/
```


动态多态

- 方法覆盖是子类的成员方法重写了父类的成员方法，重写的目的很大程度上是为了实现多态；
- 动态多态：即在运行时才能确定调用哪个方法，也称为运行时多态，也称为动态联编，也称为动绑定；
- Java中，实现动态多态有3个条件：继承、覆盖、向上转型，缺一不可。
 - “覆盖(override)方法、抽象方法和接口” 和动态联编关系紧密

主要内容

- 认识多态
 - 多态的必要性
 - 静多态和动多态
 - 多态的实现
 - 多态的优点及运行机制
- 抽象类
 - 认识抽象类
 - 理解抽象方法
 - 理解抽象类引用
- 面向对象设计模式：工厂模式
- 案例分析

Java形成动态多态必须具备以下条件：

- Java形成动态多态必须具备以下条件

... ..

```
Animal dog = new Dog();  
Doctor doctor = new Doctor();
```

```
doctor.cure(dog);
```

```
Animal lion = new Lion();
```

```
doctor.cure(lion);
```

... ..

① 必须要有继承的情况存在；

② 在继承中必须要有方法覆盖；

③ 必须由父类的引用指向派生类的实例，并且通过父类的引用调用被覆盖的方法；

- 由上述条件可以看出，**继承是实现动态多态的首要前提。**

多态的实现：继承

- Java中，一个类只能有一个直接父类，不能多继承。
- Java中，一个父类可以有多个子类，而在子类里可以覆盖父类的方法。
- 当用父类的变量去引用不同的子类，在调用这个相同的方法的时候得到的结果和表现形式就不一样了，这就是多态，相同的消息（也就是调用相同的方法）会有不同的结果

使用多态实现思路

- 编写父类
- 编写子类，子类重写（覆盖）父类方法
- 运行时，使用父类的类型，子类的对象

编码实现

继承是子类使用父类的方法，而多态则是父类使用子类的方法。（把父类当做子类来用）

```
class Printer(){  
    print(String str){}  
}
```

```
class ColorPrinter extends Printer {  
    print(String str) {  
        System.out.println("输出彩色的"+str);  
    }  
}
```

子类

```
class BlackPrinter extends Printer {  
    print(String str) {  
        System.out.println("输出黑白的"+str);  
    }  
}
```

```
public static void main(String[] args) {  
    Printer p = new ColorPrinter();  
    p.print();  
    p = new BlackPrinter();  
    p.print();  
}
```

运行

实现多态的三个要素：

1. 继承
2. 方法覆盖
3. 使用父类类型

Super.java & SubClass.java

```
1  class Sup {
2      public int x, y;
3      Sup(int a, int b) {
4          x = a;
5          y = b;
6      }
7
8      public void display() {
9          int z;
10         z = x + y;
11         System.out.println("add=" + z);
12     }
13 }
14
15 class Sub extends Sup {
16     Sub(int a, int b) {
17         super(a, b);
18     }
19
20     public void display() {
21         int z;
22         z = x * y;
23         System.out.println("product=" + z);
24     }
25 }
```

```
public class ResultDemo extends Sub
{
    ResultDemo(int x,int y)
    {
        super(x,y);
    }
    public static void main(String args[ ])
    {
        Sup num1=new Sup(7,14);
        Sub num2=new Sub(7,14);
        ResultDemo num3=new ResultDemo(7,14);
        num1.display( );
        num2.display( );
        num3.display( );
        num1=num2;
        num1.display();
        num1=num3;
        num1.display();
    }
}
```

add=21
product=98
product=98
product=98
product=98

- **方法覆盖**：方法名、参数个数、参数类型及参数顺序必须一致（函数签名）；
- 子类方法不能缩小父类方法的访问权限：
 - a) 一个默认访问权限方法可以被重写为默认权限、`protected`和`public`的；
 - b) 一个`protected`方法可以被重写为`protected`和`public`的；
 - c) 一个`public`方法只可以被重写为`public`的；

子类方法不能缩小父类方法的访问权限

```
public class Dog extends Animal {  
    public Dog(String name) {  
        super(name);  
    }  
}
```

```
protected void toHospital() {  
    this.setHealth(i: 60);  
    System.out.println(x: "Curing Dog: 打针、吃药");  
}
```

```
public class Animal{  
    String name;  
    protected String type = "Animal";  
    protected int healthValue = 99;  
  
    public Animal(String name){  
        this.name = name;  
    }  
  
    public void toHospital() {  
    }  
}
```

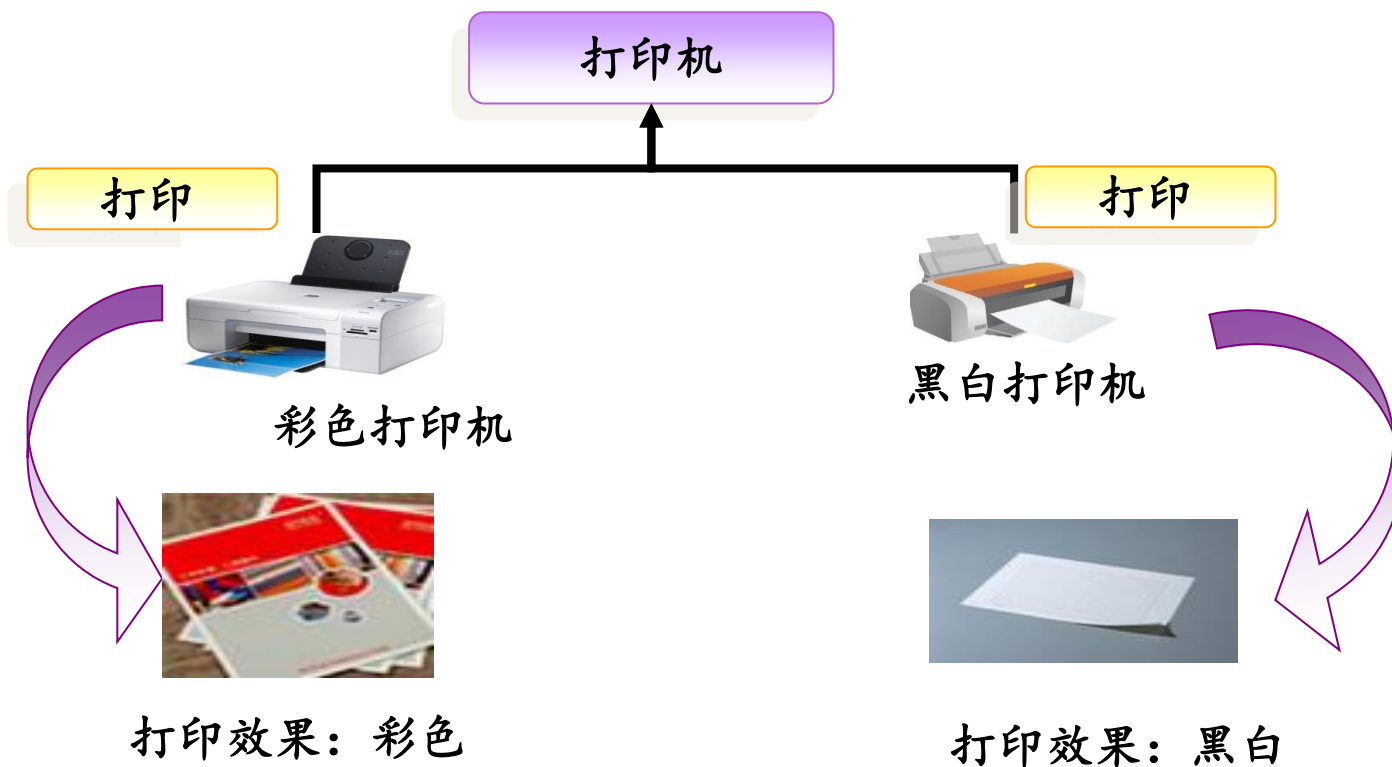
```
void cn.edu.buaa.oop.lesson_6.Dog.toHospital()
```

Cannot reduce the visibility of the inherited method from
Animal Java(67109273)

- 实现运行时多态技术的条件
 - 有一个继承层次关系；
 - 在子类中重写父类的方法, 构成方法覆盖；
 - 通过父类的引用对子类对象进行调用。

案例解析（1）：问题需求

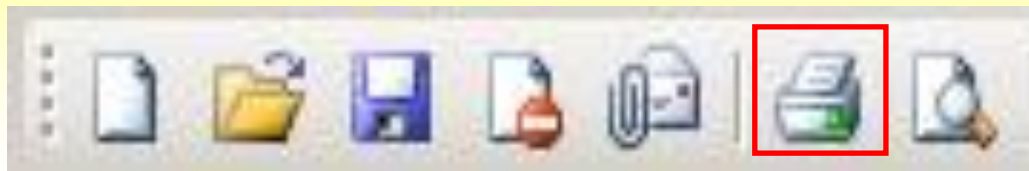
- 用多态实现打印机
 - 打印机分为黑白打印机和彩色打印机
 - 不同类型的打印机打印效果不同



分析：实现多态的流程

计算机可以连接各种打印机

无论连接何种打印机打印方法都相同



根据连接打印机不同，效果也不同

主要内容

- 认识多态
 - 多态的必要性
 - 静多态和动多态
 - 多态的实现
 - 多态的优点及运行机制
- 抽象类
 - 认识抽象类
 - 理解抽象方法
 - 理解抽象类引用
- 面向对象设计模式：工厂模式
- 案例分析

采用多态技术的优点

- 引进多态技术之后，尽管子类的对象千差万别，但都可以采用 父类引用.方法名([参数]) 统一方式来调用，在程序运行时能根据子对象的不同得到不同的结果。
- 应用程序不必为每一个派生类（子类）编写功能调用，只需要对抽象基类进行处理即可。这种“**以不变应万变**”的形式可以规范、简化程序设计，符合软件工程的“**一个接口，多种方法**”思想，可以**大大提高程序的可复用性**。
- 派生类的功能可以被基类的引用变量引用，这叫**向后兼容**，可以提高程序的**可扩充性和可维护性**。

主要内容

- 认识多态
 - 多态的必要性
 - 静多态和动多态
 - 多态的实现
 - 多态的优点及运行机制
- 抽象类
 - 认识抽象类
 - 理解抽象方法
 - 理解抽象类引用
- 面向对象设计模式：工厂模式
- 案例分析

为什么使用抽象类？

- 如果父类的方法没机会被访问调用，或者没有办法给出明确的定义。以下代码有什么问题？

```
public class Figure {  
    public void area();  
}
```

每个子类的area（）方法实现不同，父类area（）方法的实现是多余的。

抽象方法

- 可以使用抽象方法来优化
 - 抽象方法**没有方法体**

```
public abstract void area();
```

没有方法体

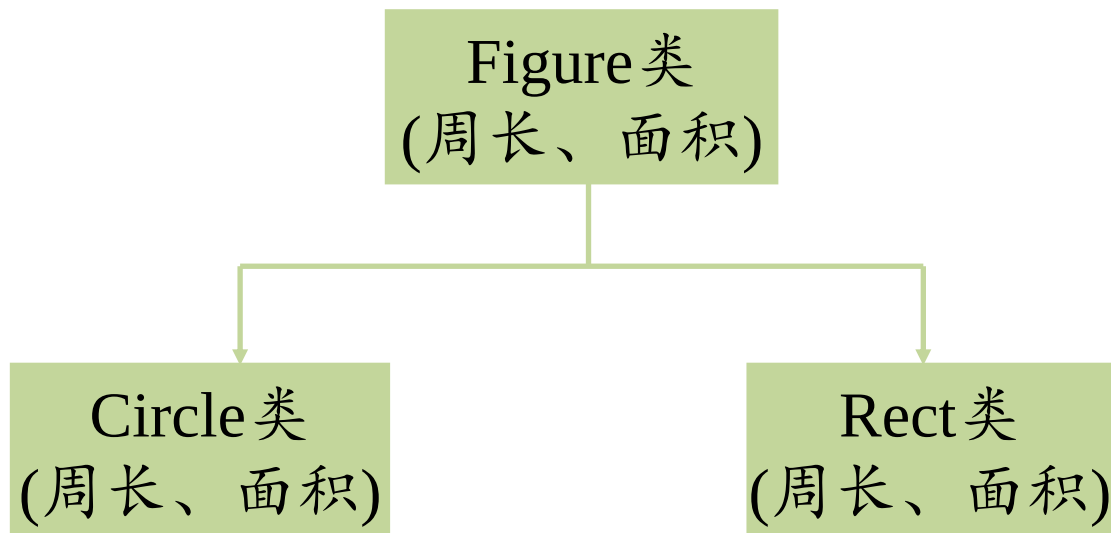
抽象类和抽象方法

- 一. 关键字`abstract` 可用来修饰方法和类，表示“尚未实现”的含义
- 二. 抽象类: 如果一个类用`abstract`修饰，则它是一个抽象类。
- 三. 抽象方法: 如果一个方法用`abstract`修饰，则它是一个抽象方法；抽象方法是没有方法体的，以一个紧跟在声明后的“;”结束。

注意: 无方法体与方法体为空是两个不同的概念。

抽象类实例

- 抽象是面向对象的一种重要方法,通过抽象我们能够设计一个更普通、更通用的类。
 - 例如: 从许许多多学生中, 抽象出Student类, 从学生、工人、农民、...抽象出Person类。
 - 下面, 我们来分析一个例子:



```
abstract class Figure { //抽象类,一般作为其它类的超类
    protected double x;

    Figure() {
    }

    Figure(double x1) {
        x = x1;
    }

    abstract public double area(); //抽象方法

    public String toString() {
        return "x=" + x;
    }
}
```

```
class Circle extends Figure {  
    public Circle(double x1) {  
        super(x1);  
    }  
  
    public double area() { //具体方法  
        return 3.1415926 * x * x;  
    }  
  
    public String toString() { //方法覆盖  
        return "圆: \t" + super.toString() + "\tarea=" + area();  
    }  
}
```

```
class Rectangle extends Figure {
```

```
    protected double y;  
    public Rectangle() {  
    };
```

```
    public Rectangle(double a, double b) {  
        super(a);  
        y = b;  
    }
```

```
    public double area() { //具体方法  
        return x * y;  
    }
```

```
    public String toString() { //方法覆盖  
        return "长方形: \t" + super.toString() +  
            "y=" + y + "\tarea=" + area();  
    }
```

```
}
```

抽象类引用

```
public class TestFigure {  
    public static void main(String args[]) {  
        Rectangle R1 = new Rectangle(10.0, 20.0);  
        Figure C1 = new Circle(10.0);  
        Figure F1 = new Rectangle(30.0, 30.0);  
        System.out.println(R1.toString());  
        System.out.println(C1.toString());  
        System.out.println(F1.toString());  
    }  
}
```

长方形:	x=10.0y=20.0	area=200.0
圆:	x=10.0	area=314.15926
长方形:	x=30.0y=30.0	area=900.0

抽象类特征

- 一. 抽象方法必须在抽象类里
- 二. 如果一个类继承自某个抽象父类，而没有具体实现抽象父类中的抽象方法，则必须定义为抽象类。
 - 抽象方法必须在子类中被实现，除非子类是抽象类
- 三. 如果一个类里有抽象的方法，则这个类就必须声明成抽象的。但一个抽象类中却可以没有抽象方法。

AbstractAnimal.java

```
public abstract class AbstractAnimal{  
    String name;  
    protected String type = "Animal";  
    protected int healthValue = 99;  
  
    public AbstractAnimal(String name){  
        this.name = name;  
    }  
  
    public abstract void enjoy();  
  
    public static void test(){  
  
    }
```

AbstractDog.java

```
public abstract class AbstractDog extends AbstractAnimal {  
    public AbstractDog(String name) {  
        super(name);  
    }  
}
```

如果一个类继承自某个抽象父类，而没有具体实现抽象父类中的抽象方法，则必须定义为抽象类。

LabradorDog.java

```
public class LabradorDog extends AbstractAnimal {  
    public LabradorDog(String name) {  
        super(name);  
    }  
  
    @Override  
    public void enjoy() {  
  
    }  
}
```

如果一个类继承自某个抽象父类，必须实现抽象父类中的抽象方法。

抽象类特征

- 抽象方法不能被private、final或static修饰。
为什么？
 - ① 抽象方法必须被子类所覆盖，如果说明为private，则外部无法访问，覆盖也无从谈起。
 - ② 若说明为static，即使不创建对象也能访问：**类名.方法名()**，这要求给出方法体，但与抽象方法的定义相矛盾。
 - ③ Final和abstract含义矛盾

抽象类与具体类的比较

抽象类	具体类
用于划分具体类	用于表示真实世界的对象
不能实例化	可以实例化
定义了未提供实现的抽象方法	不定义未提供实现的抽象方法
为自己的部分方法提供实现	为所有的方法提供实现

主要内容

- 认识多态
 - 多态的必要性
 - 静多态和动多态
 - 多态的实现
 - 多态的优点及运行机制
- 抽象类
 - 认识抽象类
 - 理解抽象方法
 - 理解抽象类引用
- 面向对象设计模式：工厂模式
- 案例分析

抽象类引用

- 抽象类是不能实例化的，但可以创建它的引用。它的作用是提供一个恰当的父类。因此一般作为其它类的超类，与final类正好相反。
- Java支持多态性，允许通过父类引用来引用子类的对象。

Animal 类

```
abstract class Animal {  
    public abstract void eat();  
    public abstract void sleep();  
}
```

```
class Giraffe extends Animal {  
    public void run() {  
        System.out.println("长颈鹿四条腿走路");  
    }  
    //覆盖  
    public void eat() {  
        System.out.println("长颈鹿在愉快的吃草");  
    }  
    //覆盖  
    public void sleep() {  
        System.out.println("长颈鹿站着睡觉");  
    }  
}
```

```
class Lion extends Animal {  
    //重写相应的方法  
    public void eat() {  
        System.out.println("狮子在吃肉");  
    }  
    //重写相应的方法  
    public void sleep() {  
        System.out.println("狮子在躺着睡觉");  
    }  
}
```

```
class Mouse extends Animal {  
    //重写相应的方法  
    public void eat() {  
        System.out.println("老鼠在吃肉");  
    }  
    //重写相应的方法  
    public void sleep() {  
        System.out.println("老鼠在躺着睡觉");  
    }  
    //添加新的方法  
    public void bore() {  
        System.out.println("老鼠在愉快地钻洞");  
    }  
}
```

AnimalTest

```
public class AnimalTest {  
    public static void main(String[] args) {  
        Animal aMouse = new Mouse();  
        Animal aGiraffe = new Giraffe();  
        Animal aLion = new Lion();  
        aMouse.eat();  
        aMouse.sleep();  
        aLion.eat();  
        aLion.sleep();  
        aGiraffe.eat();  
        aGiraffe.sleep();  
        System.out.println("-----");  
        Animal[] aArray = new Animal[3];  
        aArray[0] = aMouse;  
        aArray[1] = aGiraffe;  
        aArray[2] = aLion;  
        for (Animal i : aArray) {  
            i.eat();  
            i.sleep();  
        }  
    }  
}
```

小结

- 借助于继承和动态多态，派生类的功能可以被基类的引用变量引用，这叫**向后兼容**，从而可以提高程序的可扩充性和可维护性，程序的调用界面清楚，可读性好，并解决代码冗余。

主要内容

- 认识多态
 - 多态的必要性
 - 静多态和动多态
 - 多态的实现
 - 多态的优点及运行机制
- 抽象类
 - 认识抽象类
 - 理解抽象方法
 - 理解抽象类引用
- 面向对象设计模式：工厂模式
- 案例分析

工厂模式

- 现实生活中，原始社会自给自足（没有工厂），农耕社会小作坊（简单工厂，民间酒坊），工业革命流水线（工厂方法，自产自销），现代产业链代工厂（抽象工厂，富士康）。
- 对用户而言，给工厂需求，工厂生成出相应的产品。

工厂模式

- **工厂模式的定义**：定义一个创建产品对象的工厂接口，将产品对象的实际创建工作推迟到具体子工厂类当中。这满足**创建型模式**中所要求的“**创建与使用相分离**”的特点。
 - 简单工厂模式
 - 工厂方法模式
 - 抽象工厂模式

面向对象的形式定义计算机

使用面向对象的形式
定义计算器

```
public abstract class Operation {  
    private double value1 = 0;  
    private double value2 = 0;  
    protected abstract double getResule();  
}
```

//加法

```
public class OperationAdd extends Operation {  
    @Override  
    protected double getResult()  
    {  
        return getValue1() +  
        getValue2();  
    }  
}
```

//减法

```
public class OperationSub extends Operation {  
    @Override  
    protected double getResult()  
    {  
        return getValue1() -  
        getValue2();  
    }  
}
```

```
public static void main(String[] args) {  
    //计算两数之和  
    OperationAdd operationAdd = new OperationAdd();  
    operationAdd.setValue1(1);  
    operationAdd.setValue2(2);  
    System.out.println("sum:"+operationAdd.getResule());  
    //计算两数乘积  
    OperationMul operationMul = new OperationMul();  
    operationMul.setValue1(3);  
    operationMul.setValue2(5);  
    System.out.println("multiply:"+operationMul.getResule());  
    //计算两数之差。。。.  
}
```

简单工厂模式

- **Factory**: 即工厂类，简单工厂模式的核心部分，负责实现创建所有产品的内部逻辑；工厂类可以被外界直接调用，创建所需对象
- **Product**: 抽象类产品，它是工厂类所创建的所有对象的父类，封装了各种产品对象的公有方法，它的引入将提高系统的灵活性，使得在工厂类中只需定义一个通用的工厂方法，因为所有创建的具体产品对象都是其子类对象
- **ConcreteProduct**: 具体产品，它是简单工厂模式的创建目标，所有被创建的对象都充当这个角色的某个具体类的实例。它要实现抽象产品中声明的抽象方法

简单工厂模式

```
public class OperationFactory {  
  
    public static Operation createOperation(String operation) {  
        Operation oper = null;  
        switch (operation) {  
            case "add":  
                oper = new OperationAdd();  
                break;  
            case "sub":  
                oper = new OperationSub();  
                break;  
            case "mul":  
                oper = new OperationMul();  
                break;  
            case "div":  
                oper = new OperationDiv();  
                break;  
            default:  
                throw new UnsupportedOperationException("不支持该操作");  
        }  
        return oper;  
    }  
}
```

抽象产品类

具体产品类

简单工厂模式

```
public static void main(String[] args) {  
    Operation operationAdd = OperationFactory.createOperation("add");  
    operationAdd.setValue1(1);  
    operationAdd.setValue2(2);  
    System.out.println(operationAdd.getResule());  
}
```

工厂模式优缺点

- **可以使代码结构清晰，有效地封装变化。**在编程中，产品类的实例化有时候是比较复杂和多变的，通过工厂模式，将产品的实例化封装起来，使得调用者根本无需关心产品的实例化过程，只需依赖工厂即可得到自己想要的产品。
- **对调用者屏蔽具体的产品类。**如果使用工厂模式，调用者只关心产品的接口就可以了，至于具体的实现，调用者根本无需关心。即使变更了具体的实现，对调用者来说没有任何影响。
- **降低耦合度。**产品类的实例化通常来说是很复杂的，它需要依赖很多的类，而这些类对于调用者来说根本无需知道，如果使用了工厂方法，我们需要做的仅仅是实例化好产品类，然后交给调用者使用。对调用者来说，产品所依赖的类都是透明的。

适用场景

- 在任何需要生成复杂对象的地方
- 需要增加依赖关系时，可以考虑使用工厂模式。将会大大降低对象之间的耦合度。
- 需要系统有比较好的扩展性时，可以考虑工厂模式，不同的产品用不同的实现工厂来组装。

简单工厂模式存在的问题

- 简单工厂模式又叫作静态工厂方法模式（Static Factory Method Pattern）
- 当我们需要增加一种计算时，例如开平方。这个时候我们需要先定义一个类继承 `Operation` 类，其中实现平方的代码。除此之外我们还要修改 `OperationFactory` 类的代码，增加一个case。这显然是违背开闭原则的。可想而知对于新产品的加入，工厂类是很被动的。

怎么解决呢？

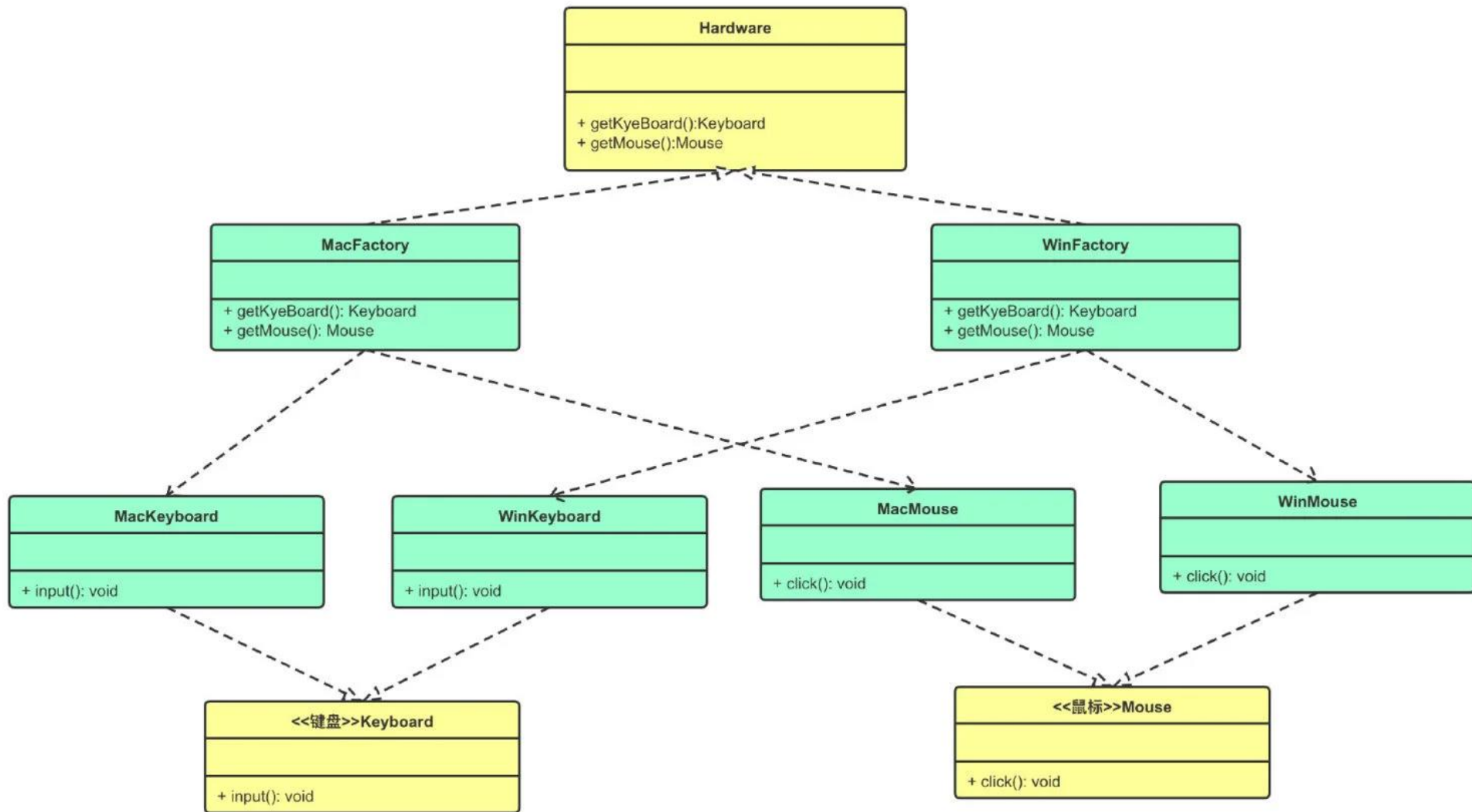
抽象工厂模式

- 抽象工厂模式（Abstract Factory Pattern）是围绕一个超级工厂创建其他工厂。
- 该超级工厂又称为其他工厂的工厂。这种类型的设计模式提供了一种创建对象的最佳方式。

抽象工厂模式包含的角色

- **AbstractFactory**: 抽象工厂，用于声明生成抽象产品的方法
- **ConcreteFactory**: 具体工厂，实现抽象工厂定义的方法，具体实现一系列产品对象的创建
- **AbstractProduct**: 抽象产品，定义一类产品对象的接口
- **ConcreteProduct**: 具体产品，通常在具体工厂里，会选择具体的产品实现，来创建符合抽象工厂定义的方法返回的产品类型的对象。

Mac、Win工厂实例



抽象工厂模式

- 抽象工厂模式只有在新增一个类型的具体产品时才需要新增工厂
- 系统中有多于一个的产品族，而每次只使用其中某一产品族
- 抽象工厂模式隔离了具体类的生成
- 当一个产品族中的多个对象被设计成一起工作时，它能够保证客户端始终只使用同一个产品族中的对象。
- 增加新的产品族很方便，无须修改已有系统，符合“开闭原则”。

主要内容

- 认识多态
 - 多态的必要性
 - 静多态和动多态
 - 多态的实现
 - 多态的优点及运行机制
- 抽象类
 - 认识抽象类
 - 理解抽象方法
 - 理解抽象类引用
- 面向对象设计模式：工厂模式
- 案例分析

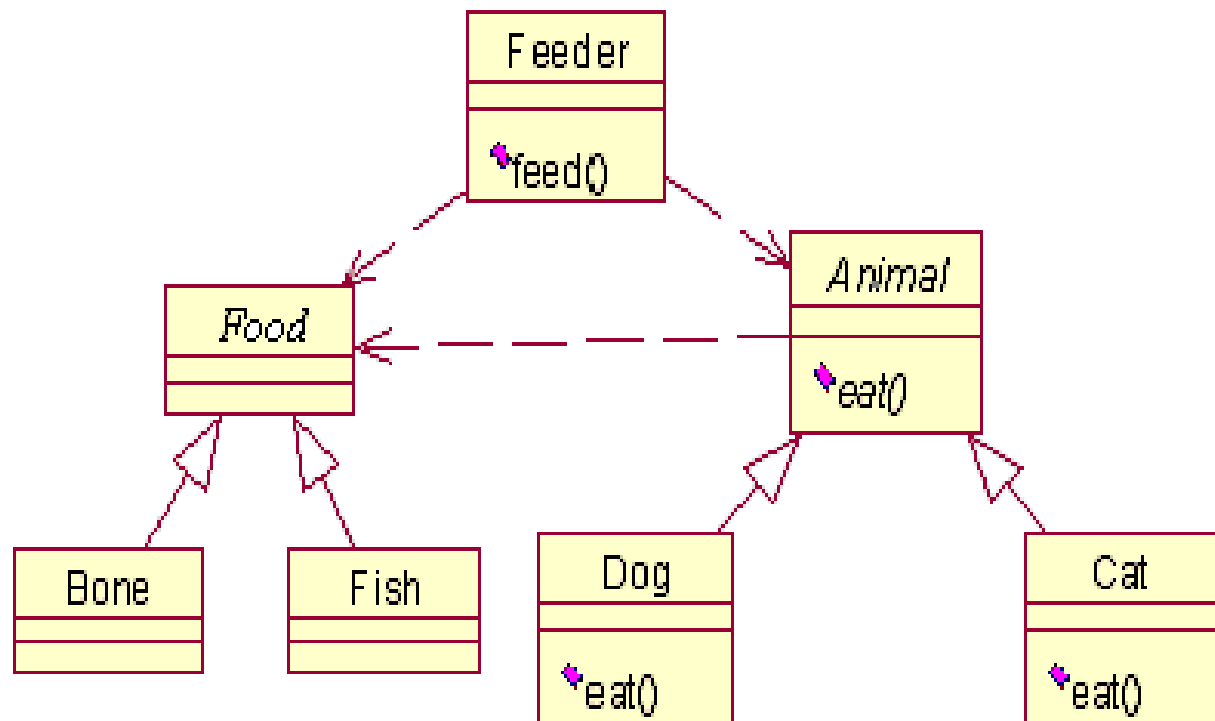
- 编写程序模拟动物园里饲养员给各种动物喂养各种不同食物的过程。

分析：

- 在这个动物园里，涉及的对象有
 - 饲养员
 - 各种不同动物
 - 以及各种不同的食物。
- 当饲养员给动物喂食时，动物发出欢快的叫声。
- 很容易抽象出3个类Feeder、Animal和Food。
- 假设只考虑猫和狗，则由Animal类派生出Cat类和Dog类
- 由Food类可以进一步派生出其子类Bone、Fish。
因为他们之间存在着明显的is-a关系。

用到的知识点

- 继承
- 多态
- 抽象类



示例代码

```
package com.buaa.adstractAnimal;  
public abstract class Food {  
    public abstract String getName();  
}
```

```
package com.buaa.adstractAnimal;  
public abstract class Animal {  
    private String name;  
  
    public Animal(String name) {  
        this.name = name;  
    }  
  
    public abstract void eat(Food food);  
    public String getName() {  
        return this.name;  
    }  
}
```


Cat类和Food类属于依赖关系

```
package com.buaa.adstractAnimal;  
public class Cat extends Animal {  
  
    public Cat(String name) {  
        super(name);  
    }  
  
    @Override  
    public void eat(Food food) {  
        System.out.println(  
            "小猫" + this.getName()  
            + "正在吃着" + food.getName()  
        );  
    }  
}
```



Dog类和Food类属于依赖关系

```
public class Dog extends Animal {  
  
    public Dog(String name) {  
        super(name);  
    }  
  
    @Override  
    public void eat(Food food) {  
        System.out.println(  
            "小狗" + this.getName()  
            + "正在啃着" + food.getName()  
        );  
    }  
}
```



```
package com.buaa.adstractAnimal;  
public class Fish extends Food{  
    @Override  
    public String getName() {  
        return "鱼";  
    }  
}
```

示例代码

```
package com.buaa.adstractAnimal;  
public class Feeder {
```

```
    private String name;  
    public Feeder(String name) {  
        this.name = name;  
    }
```



```
        public void feed(Animal animal, Food food) {  
            System.out.println(  
                "饲养员" + this.name  
                + "喂养动物" + animal.getName()  
            );  
            animal.eat(food);  
        }
```

```
    }
```

```
package com.buaa.adstractAnimal;
public class Demo {
    public static void main(String[] args) {
        Feeder feeder = new Feeder("李大壮");
        Animal a;
        Food food;

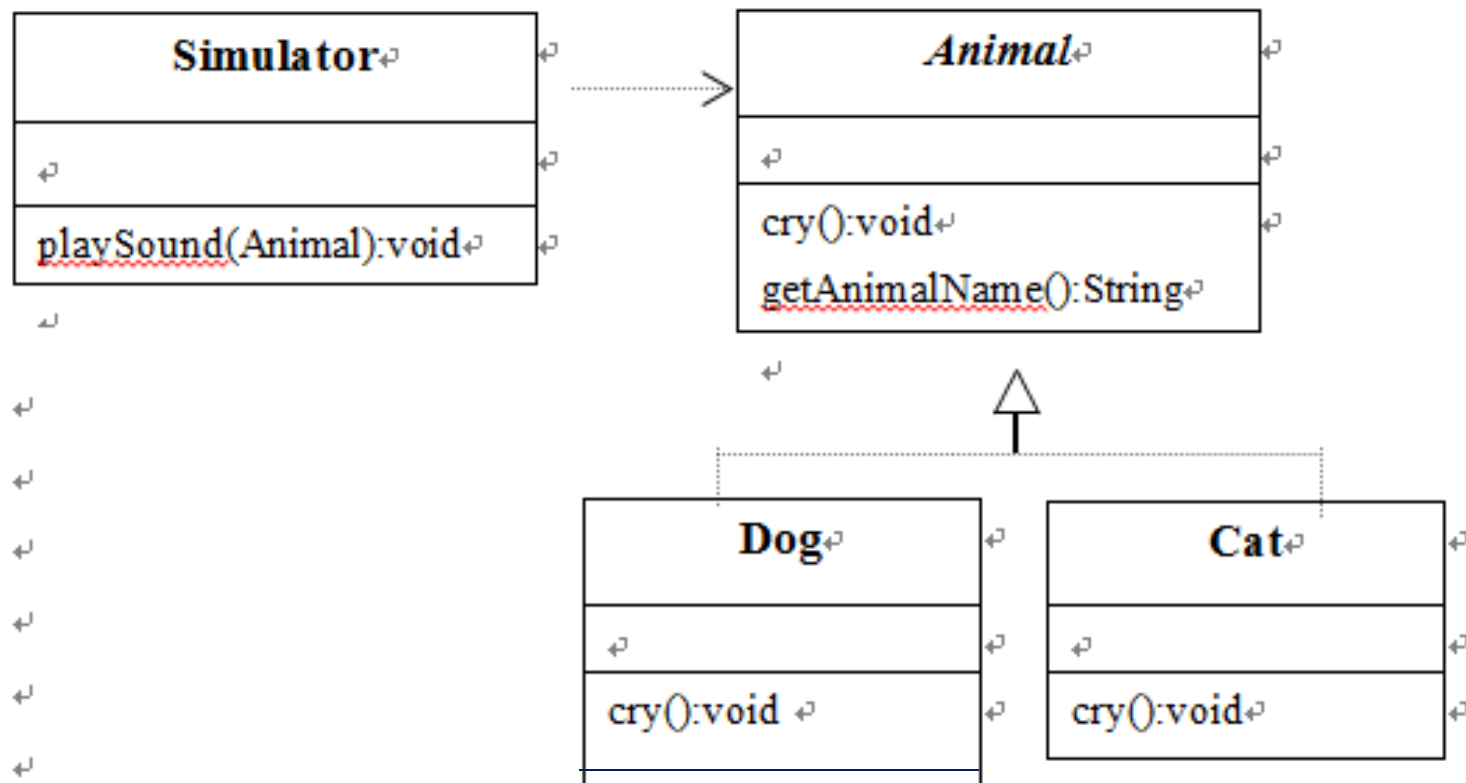
        a = new Dog("阿柴");
        food = new Bone();
        feeder.feed(a, food);

        a = new Cat("喵喵");
        food = new Fish();
        feeder.feed(a, food);
    }
}
```

案例2

- 设计一个动物声音“模拟器”，希望模拟器可以模拟许多动物的哭声。要求如下：
- 编写抽象类Animal
 - Animal抽象类有2个抽象方法cry()和getAnimalName()，即要求各种具体的动物给出自己的哭声和种类名称。
- 编写Animal类的子类：Dog，Cat类
- 编写模拟器类Simulator
 - 该类有一个playSound(Animal animal)方法，该方法的参数是Animal类型。即参数animal可以调用Animal的子类重写的cry()方法播放具体动物的声音、调用子类重写的getAnimalName()方法显示动物种类的名称。
- 编写主类Application（用户程序）

案例2



代码解析

```
public abstract class Animal {  
    public abstract void cry();  
    public abstract String getAnimalName();  
}
```

```
public class Cat extends Animal {  
    public void cry() {  
        System.out.println("喵喵...喵喵");  
    }  
    public String getAnimalName() {  
        return "猫";  
    }  
}
```


代码解析

```
public class Dog extends Animal {  
    public void cry() {  
        System.out.println("汪汪...汪汪");  
    }  
    public String getAnimalName() {  
        return "狗";  
    }  
}
```

```
public class Simulator {  
    public void playSound(Animal animal) {  
        System.out.print("现在播放" +  
            animal.getAnimalName() + "类的声音:");  
        animal.cry();  
    }  
}
```

```
public class Application {  
    public static void main(String args[]) {  
        Simulator simulator = new Simulator();  
        simulator.playSound(new Dog());  
        simulator.playSound(new Cat());  
    }  
}
```